

RAPPORT DE STAGE D'INITIATION

Filière : DATA

Conception et développement d'une plateforme web de collecte de données terrain

 **Data Ingénierie**

Bureau d'études statistiques — Maroc

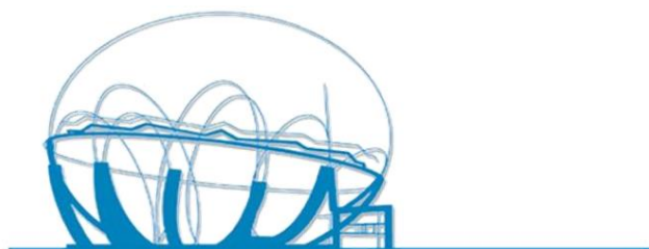
Réalisé par :

M. YOUSSEF EL KROUK

Encadré par :

M. HASSAN RIFKI

Année universitaire : 2025-2026



Remerciements

Je tiens à exprimer ma sincère gratitude à toutes les personnes qui ont contribué, de près ou de loin, au bon déroulement de ce stage de première année.

Mes remerciements s'adressent en premier lieu à **Mr Hassan Rifki**, mon encadrant au sein de Data Ingénierie, pour son accueil, sa disponibilité, ses conseils techniques et le temps qu'il a consacré à me guider tout au long de ce stage.

Mes remerciements vont aussi à l'ensemble de l'équipe de **Data Ingénierie** pour son accueil chaleureux et son esprit collaboratif, ainsi qu'à l'équipe pédagogique de **l'Institut National des Postes et Télécommunications (INPT)** pour la qualité de la formation reçue durant ce cursus.

Enfin, je remercie ma famille et mes proches pour leur soutien constant tout au long de cette expérience.

Résumé

Ce rapport présente le travail réalisé dans le cadre d'un stage d'initiation (1ère année du cycle ingénieur) effectué au sein de Data Ingénierie, bureau d'études statistiques basé au Maroc. L'objectif du stage était de concevoir et de développer une plateforme web permettant de numériser la collecte de données terrain réalisée par les enquêteurs, en remplaçant une saisie manuelle par un outil centralisé, sécurisé et accessible depuis un smartphone.

La plateforme repose sur une architecture client-serveur : un backend Django REST Framework exposant une API sécurisée par authentification JWT, une base de données PostgreSQL, et une interface frontend React consommant cette API. Deux rôles distincts — administrateur et enquêteur — disposent chacun d'un tableau de bord et de fonctionnalités adaptées à leurs besoins : saisie de fiches d'enquête, consultation et recherche des collectes, gestion des comptes enquêteurs, export des données au format CSV.

Le travail a suivi les étapes classiques de conception d'un outil logiciel : analyse des besoins, modélisation des données, développement du backend puis du frontend, containerisation avec Docker Compose, puis une phase d'amélioration de l'expérience utilisateur à travers l'ajout d'animations et de micro-interactions respectant les contraintes de performance propres à une utilisation mobile sur le terrain.

Ce stage a permis de mettre en pratique des compétences de développement full-stack, de conception d'architecture logicielle et de gestion de projet, tout en répondant à un besoin réel de digitalisation des processus de collecte statistique.

Mots-clés : Collecte de données terrain, Django REST Framework, React, PostgreSQL, API REST, authentification JWT, Docker, développement full-stack, expérience utilisateur.

Code source disponible sur GitHub : <https://github.com/youssefelkrouk-hub/django-react-survey-platform>

Abstract

This report presents the work carried out during a first-year observation internship at Data Ingénierie, a statistical consulting firm based in Morocco. The internship aimed to design and develop a web platform allowing field surveyors to digitize data collection, replacing manual paper-based entry with a centralized, secure, and mobile-friendly tool.

The platform is built on a client-server architecture: a Django REST Framework backend exposing a JWT-secured API, a PostgreSQL database, and a React frontend consuming that API. Two distinct roles — administrator and surveyor — each have a dedicated dashboard and features suited to their needs: survey form submission, browsing and searching collected records, surveyor account management, and CSV data export.

The project followed the classic stages of software tool design: requirements analysis, data modeling, backend then frontend development, containerization with Docker Compose, and a final phase dedicated to improving the user experience through animations and micro-interactions designed to respect the performance constraints of mobile field usage.

This internship provided hands-on experience in full-stack development, software architecture design, and project management, while addressing a genuine need to digitize statistical field-collection processes.

Keywords: Field data collection, Django REST Framework, React, PostgreSQL, REST API, JWT authentication, Docker, full-stack development, user experience.

Source code available on GitHub : <https://github.com/youssefelkrouk-hub/django-react-survey-platform>

Table des matières

Remerciements	1
Résumé	2
Abstract	3
1 Introduction	6
1.1 Contexte du stage	6
1.2 Motivation	6
1.3 Objectifs généraux	6
1.4 Structure du rapport	7
2 Présentation de l'entreprise	8
2.1 Data Ingénierie	8
2.2 Activités et services	8
2.3 Organisation	8
3 Contexte et objectifs du stage	9
3.1 Problématique / besoin initial	9
3.2 Objectifs du stage	9
3.3 Résultats attendus	9
4 Travaux et missions réalisées	11
4.1 Mission 1 — Analyse des besoins et conception de l'architecture	11
4.2 Mission 2 — Développement du backend (API REST)	16
4.3 Mission 3 — Développement du frontend (interface React)	17
4.4 Mission 4 — Déploiement avec Docker Compose	18
4.5 Mission 5 — Amélioration de l'expérience utilisateur (UI/UX)	20
5 Méthodologie	21
5.1 Approche suivie	21
5.2 Principales étapes de mise en œuvre	21
5.3 Outils de travail	22
6 Résultats et analyse	23
6.1 Résultats obtenus	23
6.2 Analyse	23
6.3 Captures d'écran des principales interfaces	23

7	Difficultés rencontrées et solutions	28
7.1	Gestion des permissions selon le rôle	28
7.2	Contraintes de performance sur mobile	28
7.3	Cohérence de l'identité visuelle	28
8	Compétences acquises	30
8.1	Compétences techniques	30
8.2	Compétences professionnelles et transversales	30
8.3	Compétences relationnelles (soft skills)	30
9	Conclusion	31
10	Perspectives	32
11	Annexes	33
	Annexe A — Rôles et permissions	33
	Annexe B — Services de l'environnement Docker	33

1. Introduction

1.1 Contexte du stage

Ce rapport s’inscrit dans le cadre de mon stage de première année, effectué au sein de la filière Data Science, et réalisé au sein de Data Ingénierie. Cette entreprise est un bureau d’études marocain spécialisé dans la statistique, la planification et le développement de projets¹, intervenant notamment dans les enquêtes de terrain, la sociométrie, la mise en place d’observatoires, les sondages d’opinion et le datamining. Une part importante de son activité repose ainsi sur la collecte de données auprès d’entreprises et de populations directement sur le terrain, réalisée par des équipes d’enquêteurs mobiles, afin d’alimenter les études et analyses statistiques destinées à ses clients.

Cette situation motivait la mise en place d’un outil numérique centralisé, accessible directement depuis le terrain.

1.2 Motivation

La digitalisation de la collecte de données terrain répond à plusieurs enjeux : fiabiliser la saisie, réduire les délais entre la collecte et l’exploitation des données, faciliter le suivi de l’activité des enquêteurs par les administrateurs, et permettre une consultation en temps réel des informations recueillies. Le choix d’une architecture web découplée (API + interface React) permet en outre une utilisation multiplateforme, y compris sur des smartphones Android peu puissants avec une connexion internet parfois limitée.

1.3 Objectifs généraux

- Analyser les besoins de collecte de données terrain de Data Ingénierie.
- Concevoir une architecture logicielle adaptée (backend API + frontend web).
- Développer une plateforme fonctionnelle avec gestion de rôles (administrateur / enquêteur).
- Sécuriser les accès via une authentification par jetons JWT.
- Déployer l’application dans des conteneurs Docker pour faciliter son installation.
- Améliorer l’expérience utilisateur par des animations et micro-interactions adaptées au mobile.

1. <https://www.dataingenierie.com/>

1.4 Structure du rapport

Ce rapport est organisé comme suit : après une présentation de l'entreprise d'accueil et du contexte du stage, nous détaillons les missions réalisées et la méthodologie suivie, en insistant sur les différentes étapes de conception de l'outil (analyse, modélisation, développement, tests, déploiement). Les résultats obtenus sont ensuite analysés, suivis des difficultés rencontrées et des solutions apportées. Le rapport se conclut par un bilan des compétences acquises et des perspectives d'évolution de la plateforme.

2. Présentation de l'entreprise

2.1 Data Ingénierie

Nom : Data Ingénierie

Secteur d'activité : Bureau d'études spécialisé dans la statistique, la planification et le développement de projets.

Localisation : Rabat - Agdal

2.2 Activités et services

Data Ingénierie intervient comme bureau d'études statistiques, réalisant des enquêtes et des études de terrain pour le compte de ses clients. Son activité repose sur la collecte, le traitement et l'analyse de données concernant des entreprises (identification, secteur d'activité, effectifs, localisation, coordonnées, etc.), recueillies directement auprès des entités enquêtées par des équipes d'enquêteurs mobiles.

Principaux services : Enquêtes et analyses statistiques ; enquêtes de terrain ; sociométrie et sondages d'opinion ; mise en place d'observatoires ; enquêtes de satisfaction ; systèmes d'analyse géographique et cartographie statistique ; datamining ; informatique décisionnelle et reporting ; évaluation d'impact de projets ; formation.

2.3 Organisation

L'entreprise s'organise autour de deux grandes catégories d'acteurs, qui correspondent directement aux deux rôles définis dans la plateforme développée :

- Les administrateurs, qui pilotent l'activité, supervisent les enquêteurs et exploitent les données collectées.
- Les enquêteurs de terrain, qui réalisent les visites et saisissent les informations recueillies auprès des entreprises.

3. Contexte et objectifs du stage

3.1 Problématique / besoin initial

Le processus de collecte de données terrain nécessitait un outil permettant aux enquêteurs de saisir directement, depuis un smartphone, les informations relatives aux entreprises visitées, tout en garantissant que ces données remontent immédiatement et de façon fiable vers un point central consultable par les administrateurs. Le besoin identifié portait notamment sur :

- La centralisation des fiches de collecte dans une base de données unique, évitant la dispersion des informations.
- La distinction claire entre les droits des enquêteurs (saisie de leurs propres collectes) et ceux des administrateurs (vue globale, gestion des comptes, export des données).
- Une interface simple et rapide, utilisable sur des appareils Android d'entrée de gamme avec une connexion réseau parfois instable.

3.2 Objectifs du stage

- Concevoir le modèle de données représentant les entreprises collectées, les utilisateurs et les listes de référence (secteurs, types d'activité).
- Développer une API REST sécurisée exposant les fonctionnalités nécessaires aux deux rôles.
- Développer une interface web React consommant cette API, avec une navigation adaptée à chaque rôle.
- Mettre en place l'authentification et les permissions par rôle.
- Conteneuriser l'ensemble de la solution avec Docker Compose pour simplifier le déploiement.
- Optimiser l'expérience utilisateur (animations, retours visuels, états de chargement) sans dégrader la performance sur mobile.

3.3 Résultats attendus

À l'issue du stage, la plateforme devait permettre à un enquêteur de se connecter, saisir une nouvelle fiche de collecte, consulter et rechercher l'historique de ses propres collectes ;

et à un administrateur de consulter un tableau de bord global, gérer les comptes des enquêteurs et exporter l'ensemble des données collectées au format CSV.

4. Travaux et missions réalisées

Le stage s’est organisé autour de cinq missions principales, correspondant chacune à une étape clé du cycle de conception et de développement de la plateforme.

4.1 Mission 1 — Analyse des besoins et conception de l’architecture

Contexte : Avant tout développement, il était nécessaire de clarifier les besoins fonctionnels des deux profils d’utilisateurs (administrateur et enquêteur) et de définir une architecture technique adaptée aux contraintes de l’entreprise (usage mobile, connexion limitée, sécurité des données).

Objectif : Définir le modèle de données et l’architecture logicielle qui serviront de base à l’ensemble du développement.

Travail réalisé :

- Recensement des informations à collecter pour chaque entreprise visitée (identification, secteur, localisation, coordonnées, observations de l’enquêteur).
- Définition des deux rôles applicatifs (administrateur, enquêteur) et de leurs permissions respectives.
- Choix d’une architecture client-serveur découplée : API REST côté backend, application monopage (SPA) côté frontend.
- Modélisation des entités principales : Utilisateur, Entreprise, SecteurActivité, TypeActivité.

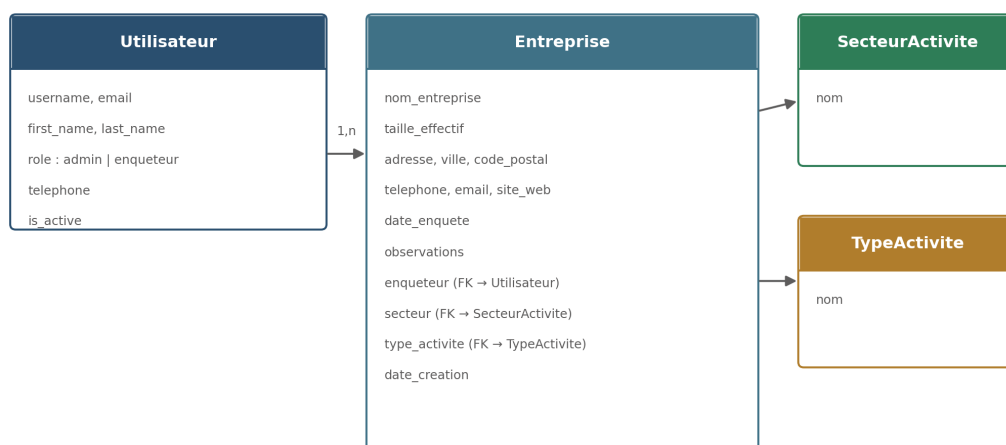
Méthodologie : Approche descendante : identification des cas d’usage, puis modélisation des entités et de leurs relations, avant validation de l’architecture technique globale.

Technologies / outils utilisés : Diagrammes de conception, Django (ORM), PostgreSQL.

Contribution de l’étudiant : Réalisation du modèle de données et des choix d’architecture, en cohérence avec les contraintes de mobilité et de sécurité exprimées par l’entreprise.

Résultats : Un modèle de données clair et normalisé, servant de référence pour le développement du backend et du frontend.

Modèle de données (simplifié) — application « collecte »



Un Utilisateur (rôle enquêteur) saisit plusieurs fiches Entreprise ; chaque fiche référence un secteur et un type d'activité issus des listes de référence.

Figure 4.1 — Modèle de données simplifié de l'application « collecte » (relations entre Utilisateur, Entreprise et listes de référence).

Diagrammes UML de conception

Afin de formaliser l'analyse et la conception présentées ci-dessus, trois diagrammes UML complémentaires ont été réalisés : un diagramme de cas d'utilisation pour représenter les interactions entre les acteurs et le système, un diagramme de classes détaillant le modèle de données, et un diagramme de séquence illustrant le déroulement des principales opérations (authentification et soumission d'une collecte).

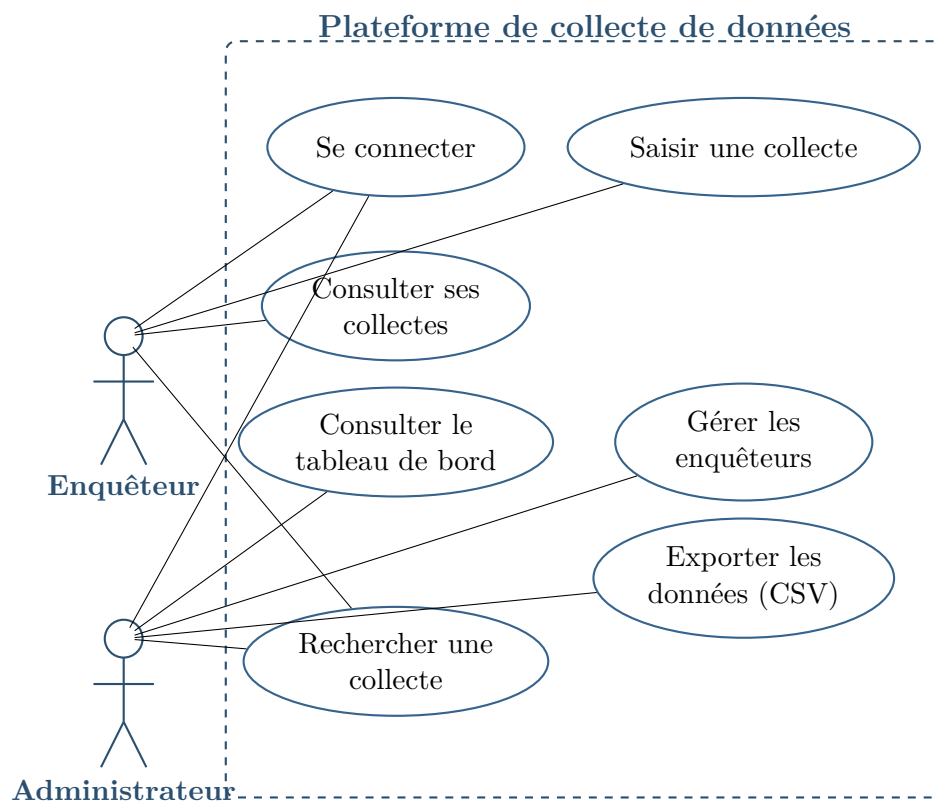


Figure 4.2 – Diagramme de cas d'utilisation

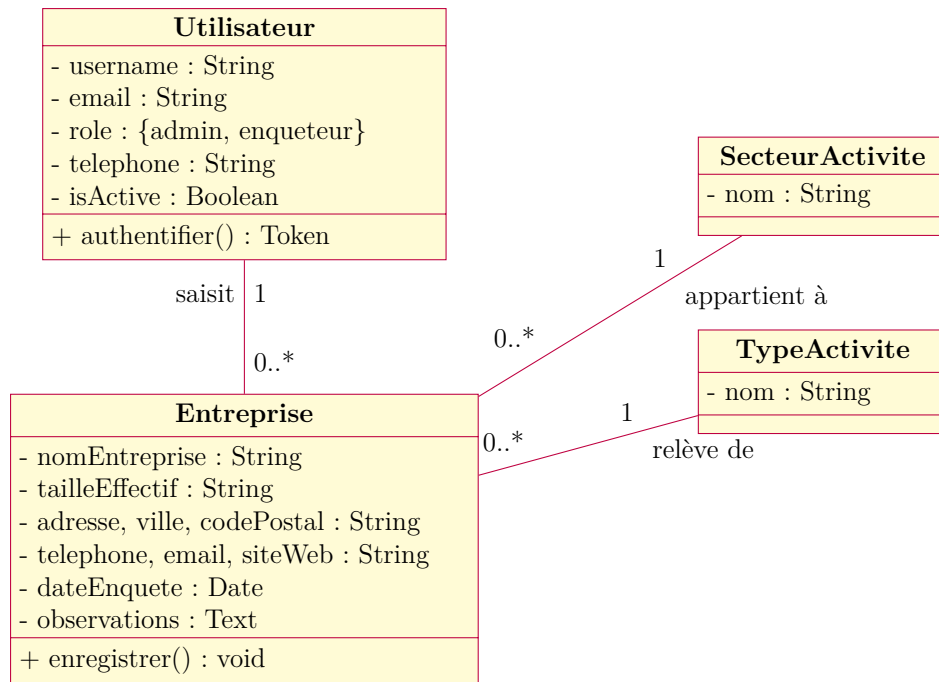


Figure 4.3 – Diagramme de classes UML

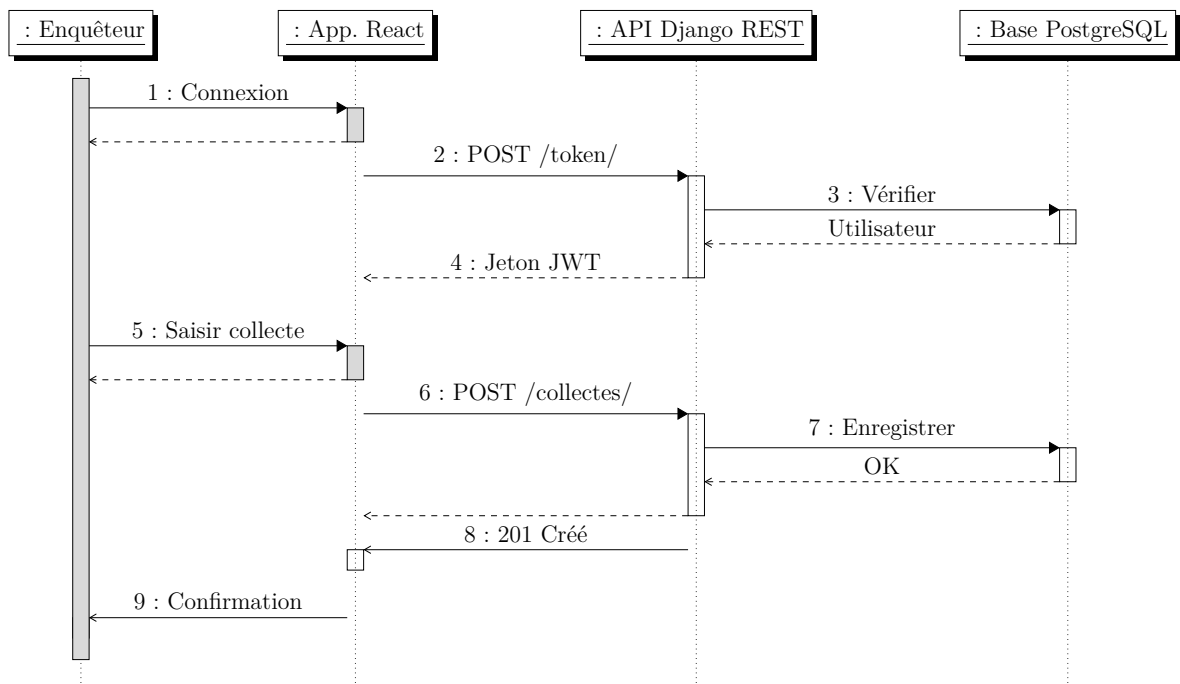


Figure 4.4 — Diagramme de séquence — Authentification et soumission d’une collecte

4.2 Mission 2 — Développement du backend (API REST)

Contexte : Le backend constitue le socle de la plateforme : il expose les données via une API sécurisée, applique les règles métier et gère les permissions selon le rôle de l'utilisateur.

Objectif : Développer une API REST complète, sécurisée et documentée, permettant au frontend d'interagir avec les données.

Travail réalisé :

- Mise en place du projet Django et de l'application « comptes » (gestion des utilisateurs) et « collecte » (gestion des fiches d'entreprise).
- Extension du modèle utilisateur standard de Django (`AbstractUser`) avec un champ « rôle » (administrateur / enquêteur) et un champ téléphone.
- Implémentation de l'authentification par jetons JWT (`django-rest-framework-simplejwt`) : connexion, rafraîchissement de session, récupération du profil connecté.
- Création des permissions personnalisées (`IsAdminUserRole`, `IsEnqueteurUserRole`) restreignant l'accès aux points d'entrée selon le rôle.
- Mise en place des `ViewSet`s et du routeur DRF pour les collectes, les enquêteurs et les listes de référence (secteurs, types d'activité).
- Développement des points d'entrée dédiés aux tableaux de bord (statistiques agrégées) et à l'export des données au format CSV.
- Attribution automatique de chaque fiche de collecte à l'enquêteur connecté, sans possibilité de modification via le formulaire (garantie de traçabilité).

Méthodologie : Développement itératif de l'API, testée au fur et à mesure via les outils intégrés à Django REST Framework, en s'assurant de la cohérence des permissions à chaque étape.

Technologies / outils utilisés : Django, Django REST Framework, `django-rest-framework-simplejwt`, PostgreSQL, Python.

Contribution de l'étudiant : Conception et implémentation des modèles de données, des sérialiseurs, des vues et des règles de permission ; mise en place du système d'authentification JWT.

Résultats : Une API REST fonctionnelle, sécurisée, exposant l'ensemble des fonctionnalités nécessaires au frontend, avec une séparation stricte des droits entre administrateurs et enquêteurs.

4.3 Mission 3 — Développement du frontend (interface React)

Contexte : Une fois l'API disponible, il fallait construire une interface utilisateur claire, adaptée au mobile, permettant à chaque rôle d'accéder rapidement à ses fonctionnalités.

Objectif : Développer une application React consommant l'API REST et proposant une expérience utilisateur adaptée au terrain.

Travail réalisé :

- Mise en place du projet React avec Vite, et intégration de Bootstrap pour la structure visuelle de base.
- Développement du contexte d'authentification (gestion du token JWT, persistance de session, redirection selon le rôle).
- Création des routes protégées, filtrant l'accès aux pages selon le rôle de l'utilisateur connecté (`ProtectedRoute`).
- Développement des tableaux de bord administrateur et enquêteur, affichant les statistiques clés (nombre d'entreprises collectées, collectes du jour, enquêteurs actifs, etc.).
- Développement du formulaire de nouvelle collecte, structuré en sections logiques (identification, localisation, coordonnées, détails de l'enquête).
- Développement de la page de consultation et de recherche des collectes, ainsi que de la page de gestion des enquêteurs (création de comptes, activation/désactivation).

Méthodologie : Développement composant par composant, en réutilisant au maximum les éléments communs (navigation, cartes, tableaux) afin de garantir la cohérence visuelle et de limiter la duplication de code.

Technologies / outils utilisés : React, React Router, Axios, Bootstrap, Bootstrap Icons, Vite.

Contribution de l'étudiant : Conception et développement de l'ensemble des pages et composants de l'interface, ainsi que de la logique de communication avec l'API (appels Axios, gestion des erreurs et des états de chargement).

Résultats : Une interface fonctionnelle et cohérente pour les deux profils d'utilisateurs, connectée en temps réel à l'API backend.

4.4 Mission 4 — Déploiement avec Docker Compose

Contexte : Pour faciliter l’installation et l’exécution de la plateforme sur différents postes (développement comme démonstration), une conteneurisation de l’ensemble de la solution était nécessaire.

Objectif : Mettre en place un environnement de déploiement reproductible, regroupant la base de données, le backend et le frontend.

Travail réalisé :

- Rédaction des fichiers `Dockerfile` pour le backend Django et pour le frontend React.
- Configuration du fichier `docker-compose.yml` définissant trois services : base de données PostgreSQL, API Django et application React.
- Mise en place d’un contrôle de santé (*healthcheck*) sur le service de base de données, garantissant que le backend ne démarre qu’une fois PostgreSQL disponible.
- Paramétrage des variables d’environnement (connexion à la base de données, URL de l’API pour le frontend).

Méthodologie : Construction incrémentale de l’environnement Docker, en testant chaque service indépendamment avant l’intégration complète via Docker Compose.

Technologies / outils utilisés : Docker, Docker Compose, PostgreSQL 16.

Contribution de l’étudiant : Rédaction et test des fichiers de configuration Docker et Docker Compose.

Résultats : Une plateforme démarrable en une seule commande (`docker compose up -build`), avec une base de données, une API et une interface web opérationnelles sur des ports dédiés.

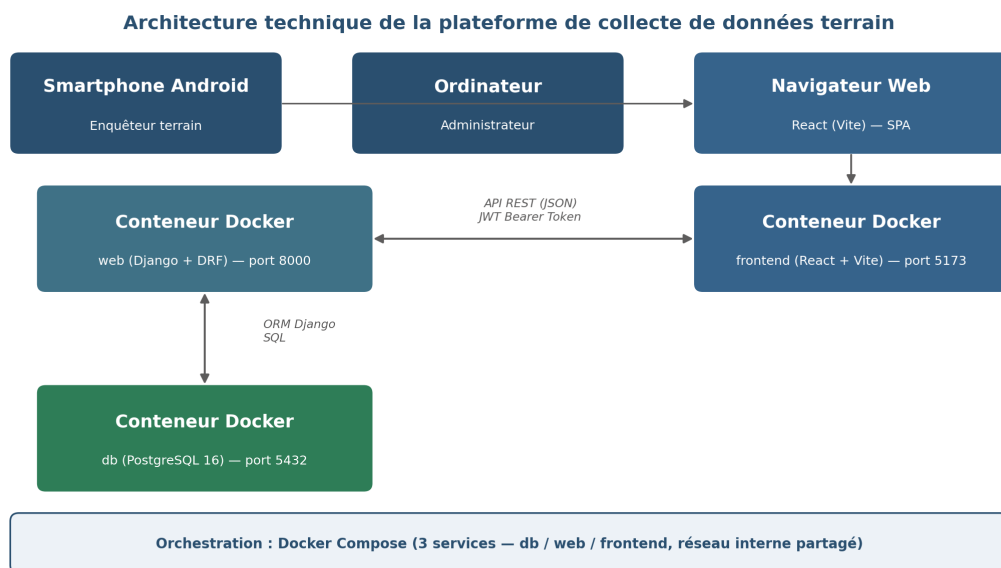


Figure 4.5 — Architecture technique de la plateforme (conteneurs Docker et flux de communication).

4.5 Mission 5 — Amélioration de l'expérience utilisateur (UI/UX)

Contexte : Une fois les fonctionnalités principales opérationnelles, une phase d'amélioration visuelle a été menée afin de rendre l'interface plus moderne et agréable à utiliser, sans altérer l'identité visuelle de Data Ingénierie ni la logique métier existante.

Objectif : Ajouter des animations et micro-interactions légères, cohérentes avec la charte graphique de l'entreprise, tout en garantissant la performance sur des appareils mobiles peu puissants.

Travail réalisé :

- Ajout de transitions douces (fondu et léger glissement) lors de la navigation entre les pages.
- Animation en cascade des cartes de statistiques et des lignes de tableaux à leur apparition.
- Amélioration des interactions sur le formulaire de collecte : mise en valeur des champs au focus, état de chargement visible sur le bouton de soumission, protection contre les double-clics.
- Animation des messages de succès et d'erreur (apparition et disparition progressives).
- Ajout de micro-interactions sur les boutons et éléments cliquables (légère variation d'échelle, transitions de couleur).
- Prise en compte de la préférence système « prefers-reduced-motion » pour désactiver les animations non essentielles lorsque l'utilisateur le souhaite.

Méthodologie : Implémentation exclusivement en CSS natif (propriétés `transform` et `opacity`), sans ajout de bibliothèque d'animation externe, afin de préserver les performances sur des connexions et des appareils limités.

Technologies / outils utilisés : CSS3 (animations, transitions, variables CSS), React.

Contribution de l'étudiant : Conception et intégration de l'ensemble des animations et micro-interactions, en veillant à ne modifier ni la charte graphique, ni la logique métier, ni les appels à l'API existants.

Résultats : Une interface perçue comme plus moderne et réactive, sans dégradation mesurable des performances, et restant accessible aux utilisateurs préférant une interface sans animation.

5. Méthodologie

5.1 Approche suivie

Le développement de la plateforme a suivi les grandes étapes classiques de la conception d'un outil logiciel, appliquées de manière itérative :

- **Analyse des besoins** : identification des informations à collecter et des profils utilisateurs.
- **Conception** : modélisation des données et définition de l'architecture technique.
- **Développement backend** : construction de l'API REST et des règles métier.
- **Développement frontend** : construction de l'interface utilisateur consommant l'API.
- **Tests et vérifications fonctionnelles** : validation manuelle des parcours utilisateurs (connexion, saisie, consultation, export).
- **Déploiement** : conteneurisation de la solution avec Docker Compose.
- **Amélioration continue** : optimisation de l'expérience utilisateur (animations, micro-interactions).

5.2 Principales étapes de mise en œuvre

Le tableau ci-dessous synthétise les technologies utilisées à chaque niveau de l'architecture.

Composant	Technologie / Rôle
Base de données	PostgreSQL 16
Backend / API	Django, Django REST Framework, djangorestframework-simplejwt
Authentification	JSON Web Tokens (JWT)
Frontend	React 18, React Router, Axios, Vite
Mise en forme	Bootstrap 5, Bootstrap Icons, CSS personnalisé
Conteneurisation	Docker, Docker Compose

5.3 Outils de travail

Le développement de la plateforme s'est appuyé sur un ensemble d'outils courants en développement web full-stack, choisis pour leur simplicité d'usage et leur bonne intégration avec la stack technique retenue (Django, React, Docker).

Éditeur de code : Visual Studio Code, avec les extensions Python, ESLint, Prettier et Docker pour faciliter respectivement le développement backend, la qualité du code frontend et la gestion des conteneurs.

Gestion de versions : Git, avec un dépôt hébergé sur GitHub , permettant de suivre l'historique des modifications et de sécuriser le code source tout au long du stage.

Test des API : Postman pour vérifier manuellement le comportement des points d'entrée de l'API avant leur intégration côté frontend.

Conteneurisation et exécution : Docker Desktop, pour construire et exécuter localement l'ensemble des services définis dans `docker-compose.yml` (base de données, backend, frontend).

Organisation du projet sous VS Code

La figure ci-dessous présente l'arborescence du projet telle qu'organisée dans l'éditeur, séparant clairement le backend Django (racine du projet) du frontend React (dossier `frontend/`).

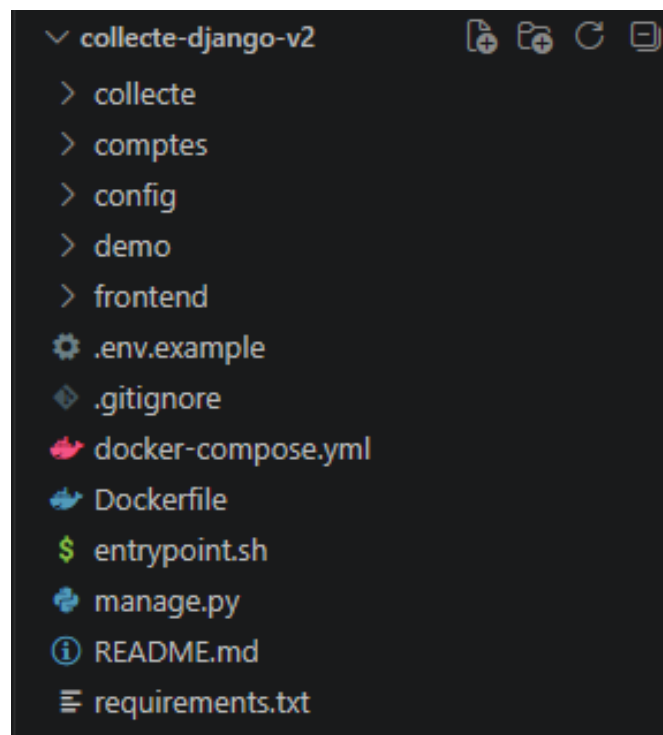


Figure 5.1 – Structure du projet dans Visual Studio Code.

6. Résultats et analyse

6.1 Résultats obtenus

Au terme du stage, la plateforme met à disposition les fonctionnalités suivantes :

- Authentification sécurisée par JWT avec redirection automatique selon le rôle de l'utilisateur.
- Tableau de bord administrateur présentant les statistiques globales (entreprises collectées, collectes du jour, enquêteurs inscrits et actifs), le classement des collectes par enquêteur et les dernières collectes enregistrées.
- Tableau de bord enquêteur présentant ses statistiques personnelles et ses dernières collectes.
- Formulaire de collecte structuré en quatre sections (identification, localisation, coordonnées, détails de l'enquête), avec validation des champs obligatoires.
- Page de consultation et de recherche des collectes, avec filtrage par entreprise ou par ville.
- Page de gestion des enquêteurs permettant à l'administrateur de créer de nouveaux comptes et d'activer ou désactiver un compte existant.
- Export de l'ensemble des données collectées au format CSV, accessible depuis le tableau de bord administrateur.
- Déploiement complet de la solution via une seule commande Docker Compose.

6.2 Analyse

L'architecture retenue — séparation stricte entre une API REST et une interface web autonome — a permis un développement parallèle et modulaire du backend et du frontend, ainsi qu'une plus grande flexibilité pour de futures évolutions (par exemple, le développement d'une application mobile native consommant la même API).

La séparation des rôles au niveau des permissions Django REST Framework garantit qu'un enquêteur ne peut ni consulter, ni modifier les données d'un autre enquêteur, ni accéder aux fonctionnalités réservées à l'administration, ce qui répond directement au besoin de sécurité et de traçabilité exprimé initialement.

6.3 Captures d'écran des principales interfaces

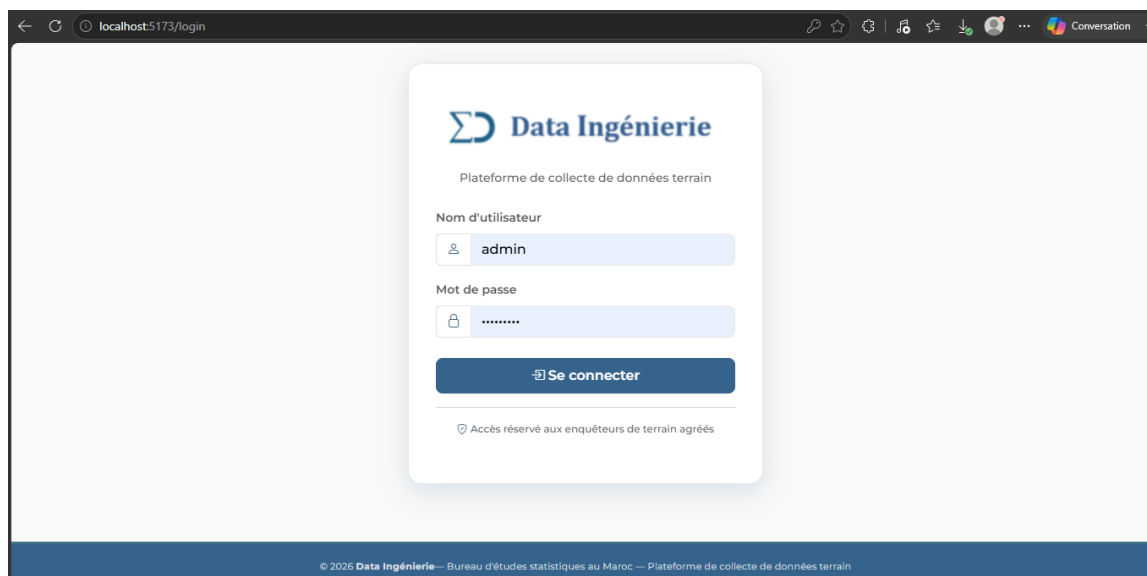


Figure 6.1 — Page de connexion — authentification par identifiant et mot de passe, sécurisée par JWT.

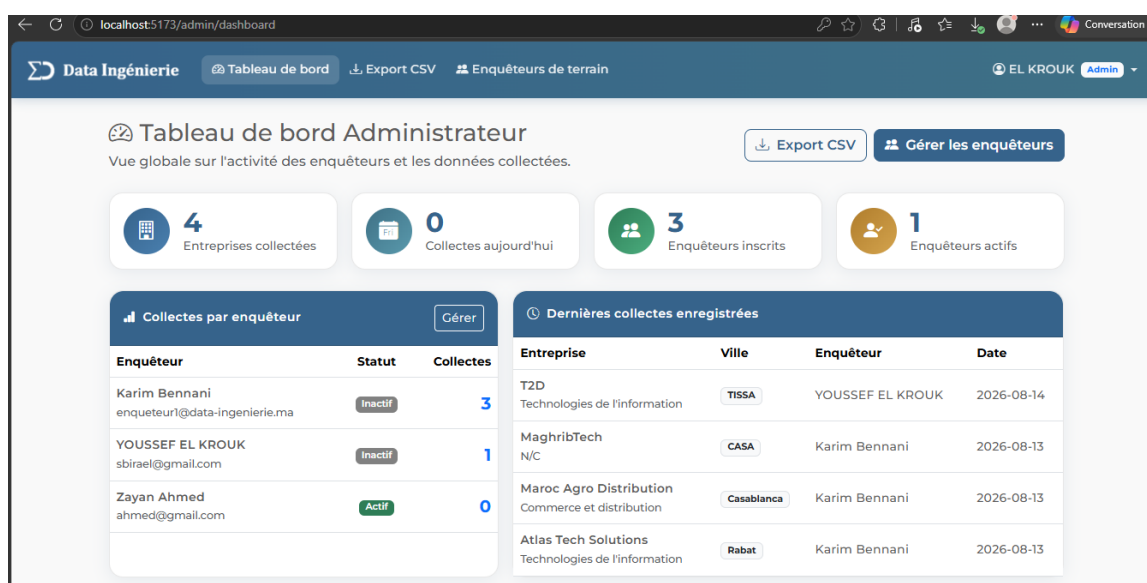


Figure 6.2 — Tableau de bord administrateur — statistiques clés et dernières collectes avec leurs enquêteurs correspondant.

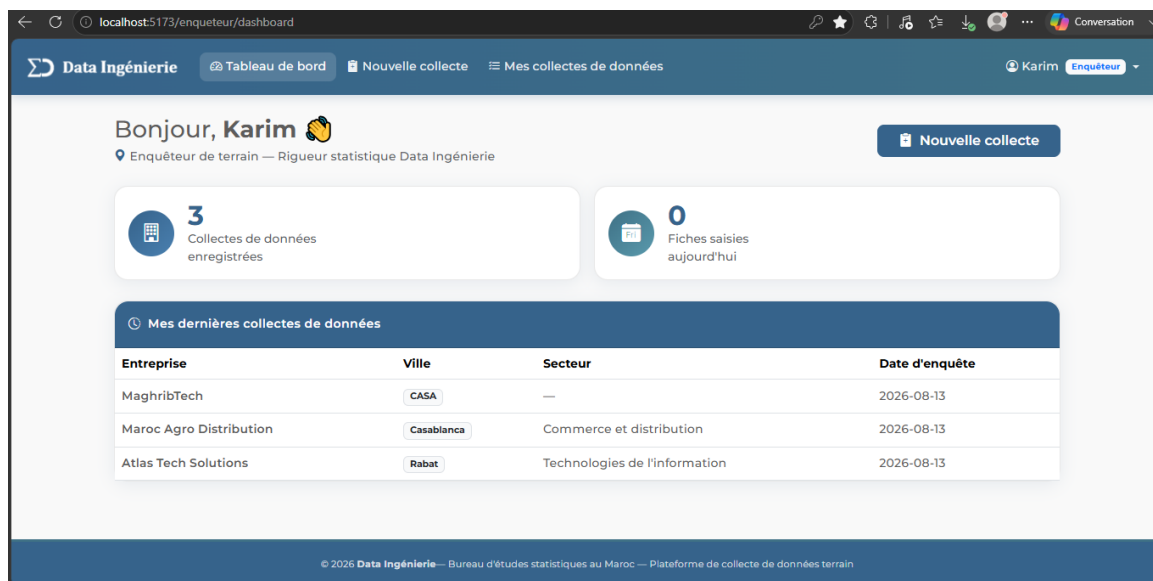


Figure 6.3 — Tableau de bord enquêteur — Vue des collectes réalisées et possibilité de lancer une nouvelle collecte.

The 'Formulaire de collecte' is structured into the following sections:

- IDENTIFICATION DE L'ENTREPRISE**
 - Raison sociale / Nom de l'entreprise *
 - Secteur d'activité and Type d'activité (dropdowns)
 - Effectif (nombre d'employés)
- LOCALISATION DE L'ENTREPRISE**
 - Adresse complète *
 - Ville * and Code postal
- COORDONNÉES DE CONTACT**
 - Téléphone and Adresse e-mail
 - Site web
- DÉTAILS DE L'ENQUÊTE TERRAIN**
 - Date de l'enquête terrain *
 - Observations de l'enquêteur terrain (text area)

At the bottom, a note states: 'Notez vos observations de rigueur statistique : contexte, fiabilité estimée des informations.' followed by a button 'Enregistrer la collecte de données'.

Figure 6.4 — Formulaire de collecte que l'enquêteur voit — saisie structurée en sections (identification, localisation, coordonnées, détails de l'enquête).

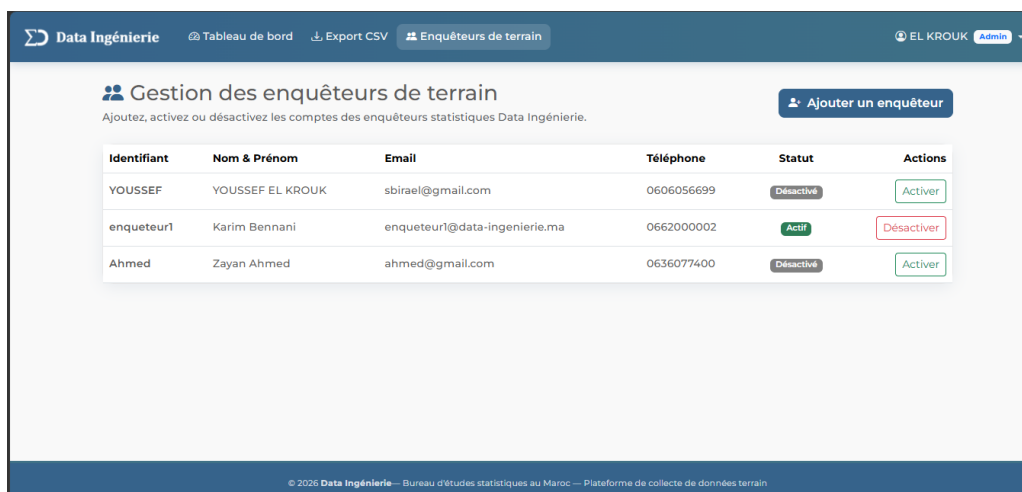


Figure 6.5 – gestion des enquêteurs de terrain en tant que administrateur — consultation des comptes, activation/désactivation et ajout de nouveaux enquêteurs

Ajouter un enquêteur de terrain

Nom d'utilisateur *

Prénom

Nom

Adresse Email

Téléphone

Mot de passe

Annuler

Créer le compte

Figure 6.6 – Formulaire d’ajout d’un enquêteur de terrain — saisie des informations personnelles et création de compte par L’Admin.

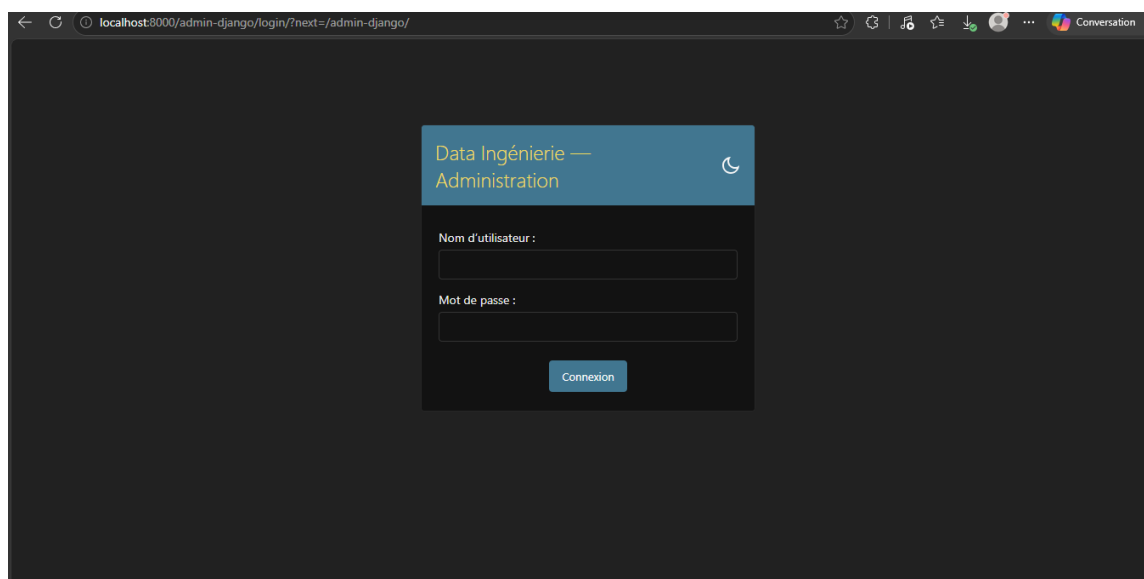


Figure 6.7 – Interface de login à l'administration pour la gestion des enquêteurs

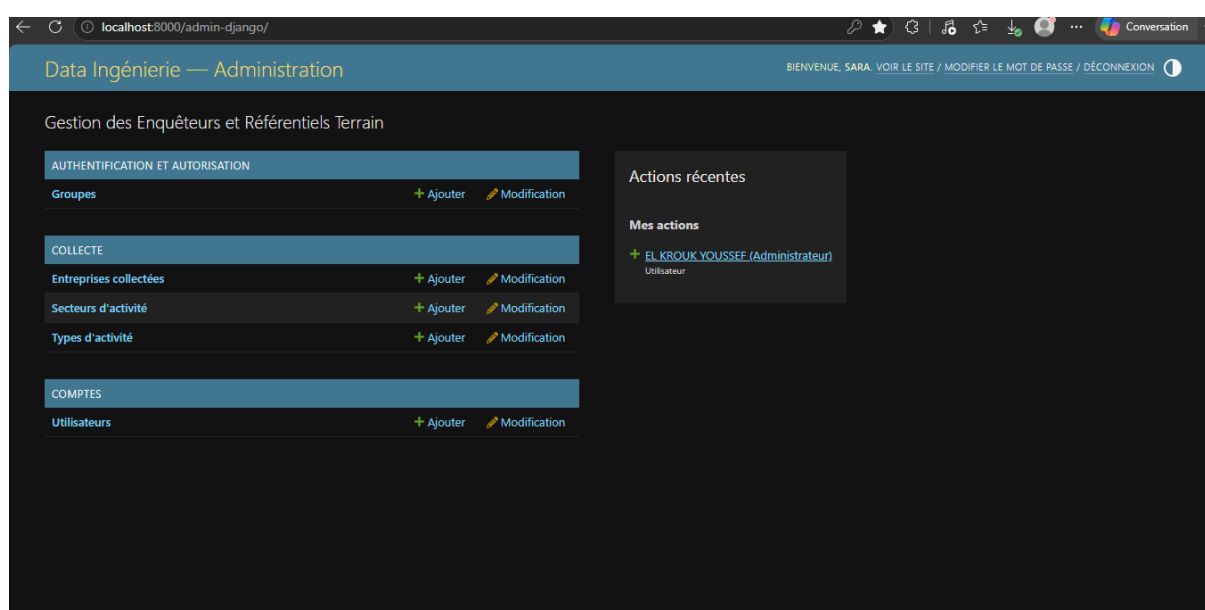


Figure 6.8 – L'interface d'administration de la plateforme Data Ingénierie constitue l'espace central de gestion des enquêtes et des comptes utilisateurs. Elle permet à l'administrateur de modifier les enquêtes existantes ainsi que les informations qu'elles contiennent, d'ajouter de nouvelles enquêtes, et de gérer les référentiels terrain (entreprises, secteurs et types d'activité). En parallèle, cette interface offre la possibilité de créer des comptes enquêteurs, de les activer ou de les désactiver, assurant ainsi un contrôle complet sur l'organisation et la supervision des activités de collecte de données.

7. Difficultés rencontrées et solutions

7.1 Gestion des permissions selon le rôle

Problème : Garantir qu'un enquêteur ne puisse accéder qu'à ses propres collectes, tout en laissant l'administrateur consulter l'ensemble des données.

Cause : L'API REST devait appliquer des règles de filtrage différentes selon le rôle sur les mêmes points d'entrée.

Solution apportée : Mise en place de classes de permission dédiées (`IsAdminUserRole`, `IsEnqueteurUserRole`) et filtrage des jeux de données au niveau des vues, en s'appuyant sur l'utilisateur authentifié par le jeton JWT.

Résultat : Une séparation fiable des données entre les rôles, vérifiée sur l'ensemble des parcours applicatifs.

7.2 Contraintes de performance sur mobile

Problème : Proposer une interface moderne et animée sans dégrader la fluidité sur des smartphones Android d'entrée de gamme ni sur une connexion internet limitée.

Cause : L'ajout d'animations peut alourdir le rendu si l'on utilise des propriétés CSS coûteuses (largeur, hauteur, position) ou des bibliothèques externes volumineuses.

Solution apportée : Utilisation exclusive de CSS natif reposant sur les propriétés `transform` et `opacity` (accélérées matériellement), sans ajout de bibliothèque d'animation externe, et prise en charge de la préférence « prefers-reduced-motion ».

Résultat : Une interface visuellement plus riche, sans impact perceptible sur les performances ni sur le poids de l'application.

7.3 Cohérence de l'identité visuelle

Problème : Moderniser l'interface sans altérer le logo, les couleurs (bleu marine et cyan) ni la structure des pages existantes.

Cause : Un risque de dérive visuelle existe lorsqu'on ajoute des effets graphiques sans cadre strict.

Solution apportée : Définition d'un système d'animation (feuille de style dédiée, variables CSS) réutilisant strictement les couleurs et rayons de bordure déjà définis dans la charte existante.

Résultat : Une interface enrichie tout en conservant fidèlement l'identité visuelle de Data Ingénierie.

8. Compétences acquises

8.1 Compétences techniques

- Conception de modèles de données relationnels et implémentation avec l'ORM Django.
- Développement d'API REST sécurisées avec Django REST Framework.
- Mise en œuvre de l'authentification par jetons JWT et de systèmes de permissions par rôle.
- Développement d'interfaces utilisateur avec React (composants, routage, gestion d'état, appels API asynchrones).
- Conteneurisation d'une application multi-services avec Docker et Docker Compose.
- Écriture d'animations et de micro-interactions CSS performantes, respectant les contraintes d'accessibilité.

8.2 Compétences professionnelles et transversales

- Analyse d'un besoin métier et traduction en spécifications techniques.
- Organisation du travail en missions successives, du besoin jusqu'au déploiement.
- Documentation et structuration du code pour en faciliter la maintenance.

8.3 Compétences relationnelles (soft skills)

- Communication avec les encadrants pour clarifier les besoins et valider les choix techniques.
- Autonomie dans la résolution de problèmes techniques.
- Rigueur et sens du détail, notamment dans le respect de l'identité visuelle de l'entreprise.

9. Conclusion

Ce stage de fin d'année a permis de concevoir et de développer, de bout en bout, une plateforme web de collecte de données terrain répondant à un besoin concret de Data Ingénierie : remplacer une collecte manuelle par un outil numérique centralisé, sécurisé et accessible depuis le terrain.

Le projet a suivi l'ensemble des étapes de conception d'un outil logiciel — analyse des besoins, modélisation des données, développement backend et frontend, déploiement conteneurisé, puis amélioration de l'expérience utilisateur — offrant une vision complète du cycle de vie d'une application web moderne.

Sur le plan technique, ce stage a permis de mettre en pratique et de consolider des compétences en développement full-stack (Django REST Framework et React), en conception d'architecture logicielle et en déploiement applicatif. Sur le plan professionnel, il a été l'occasion de travailler en autonomie sur un projet réel, en tenant compte de contraintes concrètes (rôles utilisateurs, sécurité, performance mobile) et de l'identité visuelle d'une entreprise existante.

10. Perspectives

Plusieurs évolutions pourraient être envisagées pour prolonger ce travail :

- Mise en place d'un mode de fonctionnement hors-ligne (*offline-first*), permettant à l'enquêteur de saisir une fiche sans connexion internet et de la synchroniser ultérieurement.
- Ajout de la géolocalisation automatique lors de la saisie d'une fiche de collecte.
- Enrichissement des tableaux de bord avec des graphiques statistiques (répartition par secteur, évolution dans le temps).
- Export des données dans d'autres formats (Excel, PDF) en complément du CSV existant.
- Mise en place de tests automatisés (backend et frontend) et d'une chaîne d'intégration continue (CI/CD).
- Ajout de notifications pour informer les administrateurs de nouvelles collectes en temps réel.

11. Annexes

Annexe A — Rôles et permissions

Extrait des classes de permission définies côté backend, garantissant la séparation des accès entre administrateurs et enquêteurs :

IsAdminUserRole : accès réservé aux utilisateurs dont le rôle est « admin » (ou superutilisateur).

IsEnqueteurUserRole : accès réservé aux utilisateurs dont le rôle est « enqueteur ».

Annexe B — Services de l’environnement Docker

Service	Description
db	PostgreSQL 16 — port 5432, volume persistant, contrôle de santé
web	API Django REST Framework — port 8000
frontend	Application React (Vite) — port 5173